PRACTICAL-1

1. **Linear and Binary Search.**

Answer:

```
import java.util.Arrays;
import java.util.Random;
public class Main {

public static int binarySearch(int[] arr, int x) {
int l = 0, r = arr.length - 1;
while (l <= r) {
int m = l + (r - l) / 2;

// Check if x is present at mid
if (arr[m] == x)
return m;

// If x greater, ignore left half
if (arr[m] < x)
l = m + 1;

// If x is smaller, ignore right half
else
```

```
r = m - 1;
}

// If we reach here, the element was not present
return -1;
}

public static void main(String[] args) {
int[] sizes = {100, 1000, 10000, 100000, 1000000}; // Array sizes to test
int numTrials = 100; // Number of trials for each array size

for (int size : sizes) {
long totalTime = 0;

for (int trial = 0; trial < numTrials; trial++) {
int[] arr = new int[size];
Random rand = new Random();
for (int i = 0; i < size; i++) {
arr[i] = rand.nextInt(size);
}
Arrays.sort(arr); // Binary search requires a sorted array

long startTime = System.nanoTime();
int target = rand.nextInt(size); // Generate a random target
binarySearch(arr, target);
```

```
long endTime = System.nanoTime();


totalTime += (endTime - startTime);
}


double averageTime = (double) totalTime / numTrials;
System.out.println("Array size: " + size + ", Average time: " + averageTime / 1000000 + " ms");
}
}
}
```

Screenshot of Output:

```
Array size: 100, Average time: 0.00213621 ms
Array size: 1000, Average time: 9.068300000000001E-4 ms
Array size: 10000, Average time: 0.00480351 ms
Array size: 100000, Average time: 5.569700000000001E-4 ms
```

PRACTICAL-2

2. Sorting

Answer:

```
import java.util.Random;

public class Main {

    public static void main(String[] args) {
        int size = 900;
        int[] arr = new int[size];
        Random rand = new Random();

        // Bubble Sort
        long startTime = System.nanoTime();
        bubbleSort(arr, size, rand);
        long endTime = System.nanoTime();
        double bubbleSortTime = (endTime - startTime) / 1e9; // Convert nanoseconds to seconds
        System.out.println("Bubble Sort Time: " + bubbleSortTime + " seconds");

        // Selection Sort
        startTime = System.nanoTime();
        selectionSort(arr, size, rand);
        endTime = System.nanoTime();
        double selectionSortTime = (endTime - startTime) / 1e9;
        System.out.println("Selection Sort Time: " + selectionSortTime + " seconds");

        // Insertion Sort
        startTime = System.nanoTime();
        insertionSort(arr, size, rand);
        endTime = System.nanoTime();
        double insertionSortTime = (endTime - startTime) / 1e9;
        System.out.println("Insertion Sort Time: " + insertionSortTime + " seconds");
    }
```

```
public static void bubbleSort(int[] arr, int size, Random rand) {
    for (int i = 0; i < size; i++) {
        arr[i] = rand.nextInt(100);
    }

    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

public static void selectionSort(int[] arr, int size, Random rand) {
    for (int i = 0; i < size; i++) {
        arr[i] = rand.nextInt(100);
    }

    for (int i = 0; i < size - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Swap arr[i] and arr[minIndex]
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

public static void insertionSort(int[] arr, int size, Random rand) {
```

```
        for (int i = 0; i < size; i++) {
            arr[i] = rand.nextInt(100);
        }

        for (int i = 1; i < size; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j = j - 1;
            }
            arr[j + 1] = key;
        }
    }
}
```

Screenshot of Output:

```
Bubble Sort Time: 0.00604719 seconds
Selection Sort Time: 0.00422289 seconds
Insertion Sort Time: 0.004044771 seconds
```

PRACTICAL-3

3. Factorial using Recursion.

Answer:

```
import java.util.Scanner;

public class Factorial {

    public static long iterativeFactorial(int n) {
        long result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    public static long recursiveFactorial(int n) {
        if (n == 0) {
            return 1;
        } else {
            return n * recursiveFactorial(n - 1);
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number to find factorial: ");
        int n = scanner.nextInt();

        // Iterative approach
        long startTime = System.nanoTime();
        long iterativeResult = iterativeFactorial(n);
        long endTime = System.nanoTime();
        double iterativeTime = (endTime - startTime) / 1e9; // Convert nanoseconds to seconds

        System.out.println("Time for iterative: " + iterativeTime + " seconds");
```

```
        System.out.println("Factorial of " + n + " is " + iterativeResult);

        // Recursive approach
        startTime = System.nanoTime();
        long recursiveResult = recursiveFactorial(n);
        endTime = System.nanoTime();
        double recursiveTime = (endTime - startTime) / 1e9;

        System.out.println("Time for recursive: " + recursiveTime + " seconds");
        System.out.println("Factorial of " + n + " is " + recursiveResult);
    }
}
```

Screenshot of Output:

```
Enter a number to find factorial: 14

Time for iterative is 2e-06

Factorial of 14 is 8.71783e+10

Time for recursive is 1e-06

Factorial of 14 is -2.14748e+09
```

PRACTICAL-4

4. **Heap tree.**

Answer:

```
import java.util.Scanner;

public class Main {

    static int size;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter size of array: ");
        size = scanner.nextInt();

        // Allocate memory for the array
        int[] a = new int[size];

        System.out.println("Enter " + size + " elements in array:");
        for (int i = 0; i < size; i++) {
            // Store elements in the array
            a[i] = scanner.nextInt();
        }

        heapSort(a);

        System.out.println("\nSorted heap is:");
        for (int i = 0; i < size; i++) {
            System.out.print(a[i] + " ");
        }

        System.out.println("\n\n");

        int y = (int) (Math.log(size + 1) / Math.log(2)) + 1;
        int l = 0;
```

```
    for (int j = 0; j <= (int) (Math.log(size + 1) / Math.log(2)) + 1; j++) {
        System.out.println();

        for (int p = y; p >= 0; p--) {
            System.out.print("  ");
        }

        y--;

        for (int i = 0; (i < Math.pow(2, j) && l < size); i++) {
            // Access elements correctly
            System.out.print(a[size - 1 - l] + " ");
            l++;
        }

        System.out.println();
    }
}

public static void heapSort(int[] b) {
    int heapSize = size;
    buildMaxHeap(b);

    for (int i = size - 1; i >= 0; i--) {
        int temp = b[0];
        b[0] = b[i];
        b[i] = temp;
        heapSize--;
        maxHeapify(b, 0, heapSize);
    }
}

public static void buildMaxHeap(int[] c) {
    int heapSize = size;

    for (int i = size / 2 - 1; i >= 0; i--) {
        maxHeapify(c, i, heapSize);
    }
```

```
    }

    public static void maxHeapify(int[] d, int a, int heapSize) {
        int largest = a;
        int left = 2 * a + 1;
        int right = 2 * a + 2;

        if (left < heapSize && d[left] > d[largest]) {
            largest = left;
        }

        if (right < heapSize && d[right] > d[largest]) {
            largest = right;
        }

        if (largest != a) {
            int temp = d[a];
            d[a] = d[largest];
            d[largest] = temp;
            maxHeapify(d, largest, heapSize);
        }
    }
}
```

Screenshot of Output:

```
Enter size of array: 4
Enter 4 elements in array:
3
54
5
3

Sorted heap is:
3 3 5 54



        54

      5 3

    3
```

PRACTICAL-5

5. Knapsack problem Using Dynamic Programming.

Answer:

```
import java.util.Scanner;
import java.util.ArrayList;
import java.util.List;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter number of objects: ");
        int n = scanner.nextInt();

        System.out.print("Enter maximum weight, knapsack can carry: ");
        int W = scanner.nextInt();

        int[] v = new int[n + 1]; // Values
        int[] w = new int[n + 1]; // Weights
        int[][] k = new int[n + 1][W + 1]; // DP table

        System.out.println("Enter weights of objects:");
        for (int i = 1; i <= n; i++) {
            System.out.print("w[" + i + "] = ");
            w[i] = scanner.nextInt();
        }

        System.out.println("Enter profits/values of objects:");
        for (int i = 1; i <= n; i++) {
            System.out.print("v[" + i + "] = ");
            v[i] = scanner.nextInt();
        }
```

```
// Calculate the maximum profit for each subproblem
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= W; j++) {
        if (w[i] <= j) {
            k[i][j] = Math.max(v[i] + k[i - 1][j - w[i]], k[i - 1][j]);
        } else {
            k[i][j] = k[i - 1][j];
        }
    }
}

// Print the k matrix (optional)
System.out.println("\nP\tW\ti\t");
for (int i = 0; i <= W; i++) {
    System.out.print(i + "\t");
}
System.out.println();
for (int i = 0; i <= n; i++) {
    System.out.print(v[i] + "\t" + w[i] + "\t" + i + "\t");
    for (int j = 0; j <= W; j++) {
        System.out.print(k[i][j] + "\t");
    }
    System.out.println();
}

// Reconstruct the solution (find which items are included)
int i = n, j = W;
List<Integer> includedItems = new ArrayList<>();

while (i > 0 && j > 0) {
    if (k[i][j] != k[i - 1][j]) {
        includedItems.add(i);
        j -= w[i];
    }
    i--;
}
```

```
        System.out.print("\nThe individual weights are: ");
        for (int item : includedItems) {
            System.out.print(item + " ");
        }
        System.out.println();

        // Calculate total weight and profit
        int totalWeight = 0;
        int totalProfit = 0;
        for (int item : includedItems) {
            totalWeight += w[item];
            totalProfit += v[item];
        }

        System.out.println("Total weight to be carried in knapsack is: " + totalWeight);
        System.out.println("Total profit in knapsack is: " + totalProfit);
    }
}
```

Screenshot of Output:

```
Enter number of objects: 5
Enter maximum weight, knapsack can carry: 3
Enter weights of objects:
w[1] = 4
w[2] = 5
w[3] = 6
w[4] = 5
w[5] = 4
Enter profits/values of objects:
v[1] = 3
v[2] = 5
v[3] = 6
v[4] = 4
v[5] = 7

P    W    i
0    1    2    3
0    0    0    0    0    0    0
3    4    1    0    0    0    0
5    5    2    0    0    0    0
6    6    3    0    0    0    0
4    5    4    0    0    0    0
7    4    5    0    0    0    0
```

PRACTICAL-6

6. Implement Matrix Multiplication.

Answer:

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter number of matrices: ");
        int n = scanner.nextInt();

        int[] p = new int[n + 1];
        int[][] m = new int[n + 1][n + 1];
        int[][] s = new int[n + 1][n + 1];

        System.out.println("Enter P array values:");
        for (int i = 0; i <= n; i++) {
            System.out.print("p[" + i + "] = ");
            p[i] = scanner.nextInt();
        }

        // Initialize m and s matrices
        for (int i = 0; i <= n; i++) {
            for (int j = 0; j <= n; j++) {
                m[i][j] = 0;
                s[i][j] = 0;
            }
        }

        // Calculate minimum cost of multiplication
        for (int l = 2; l <= n; l++) {
            for (int i = 1; i <= n - l + 1; i++) {
                int j = i + l - 1;
```

```
            m[i][j] = Integer.MAX_VALUE;
            for (int k = i; k < j; k++) {
                int q = m[i][k] + m[k + 1][j] + (p[i - 1] * p[k] * p[j]);
                if (q < m[i][j]) {
                    m[i][j] = q;
                    s[i][j] = k;
                }
            }
        }
    }

    System.out.println("\n\nm Table\n\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            System.out.print(m[i][j] + "\t");
        }
        System.out.println();
    }

    System.out.println("\n\n\ns Table\n\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            System.out.print(s[i][j] + "\t");
        }
        System.out.println();
    }

    System.out.println("\n\nThe solution is: ");
    printOptimalParenthesis(s, 1, n);
    System.out.println("\n\n");

    scanner.close();
}

private static void printOptimalParenthesis(int[][] s, int i, int j) {
    if (i == j) {
        System.out.print("A" + i);
    } else {
```

```
            System.out.print("(");
            printOptimalParenthesis(s, i, s[i][j]);
            printOptimalParenthesis(s, s[i][j] + 1, j);
            System.out.print(")");
        }
    }
}
```

Screenshot of Output:

```
Enter number of matrices: 2
Enter P array values:
p[0] = 2
p[1] = 3
p[2] = 4


m Table


0   24
0   0


s Table


0   1
0   0


The solution is:
(A1A2)
```